

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND
MICROELECTRONICS

Laboratory Work 4

*Dynamic Regular Expression Generator with Universal
Variant Coverage*

Course:

Formal Languages and Finite
Automata

Author:

Lucian-Adrian Gavril

Variant 4:

$(S|T)(U|V)W^*Y^+Z^4$

$L(M|N)O^3P^*Q(2|3)$

$R^*S(T|U|V)W(X|Y|Z)^2$

Contents

1 Abstract	3
2 Introduction	3
3 Assignment Interpretation and Objectives	4
4 Theoretical Background	4
4.1 Regular Expressions as Descriptions of Regular Languages	4
4.2 Why an AST is the Right Internal Form	5
4.3 Bounded Generation as an Operational Strategy	5
5 System Architecture	6
6 Parsing and Normalization	7
6.1 Normalization Layer	7
6.2 Recursive-Descent Strategy	7
7 Generation Algorithms	8
7.1 Deterministic Enumeration	8
7.2 Random Trace Sampling	9
8 Independent Validation and Visualization	9
8.1 Validation Through an Independent Backend	9
8.2 Visualization Layer	10
9 Universal Variant Coverage	10
9.1 Canonicalized Expressions	10
9.2 Variant Task Images	11
9.3 Variant-by-Variant Discussion	11
10 Empirical Analysis Across Variants	11
11 Representative Generated Outputs	12
12 Reproducible Evidence Bundle	13
13 Benchmark Measurements	14

14 Complexity, Safety, and Search Behavior	15
15 Correctness Argument	16
15.1 Claim 1: Structure Preservation During Parsing	16
15.2 Claim 2: Soundness of Bounded Generation	16
15.3 Claim 3: Independent Validation of Accepted Outputs	16
15.4 What This Correctness Story Does and Does Not Guarantee	17
16 Verification and Testing	17
16.1 Terminal Transcript Evidence	18
17 Threats to Validity and Reproducibility Considerations	18
17.1 Handwritten-to-Canonical Interpretation Risk	19
17.2 Environment-Dependent Benchmark Results	19
17.3 Bounded Enumeration vs Full Language Coverage	19
17.4 Repository-Level Reproducibility	19
18 Discussion and ML/System Analogy	19
19 Limitations and Future Work	20
20 Conclusion	20
A Appendix A - Parser Source Listing	21
B Appendix B - Generator Source Listing	25
C Appendix C - Visualization Source Listing	30
D Appendix D - Test Suite Listing	35
E Appendix E - Generated Terminal Transcript	37
F Appendix F - Evidence Summary	39
G Appendix G - Analysis Module Listing	41
H Appendix H - Analysis Test Listing	48
I References	50

1 Abstract

This laboratory work addresses the problem of dynamically interpreting regular expressions and generating valid strings that conform to them under the constraints of a formal-languages assignment. The goal is not to construct handcrafted output templates for one particular exercise, but rather to engineer a reusable system that can parse multiple regular expressions, convert them into an internal abstract syntax tree, generate valid words, validate those words independently, and explain the generation process step by step.

The implemented solution covers all four assignment variants and all twelve expressions represented by those variants. It includes a normalization layer for handwritten notation, a recursive-descent parser, a bounded generator for languages with potentially infinite repetition, an independent validator based on Python regular expressions, and a visualization subsystem that exports Mermaid-compatible diagrams and generation traces. This transforms the laboratory work from a narrow script into a small, well-structured symbolic language engine.

From an academic perspective, the project demonstrates several important ideas at once: the translation from syntax to structure, the operational interpretation of repetition under bounded conditions, and the role of independent verification in establishing trust in generated results. From an engineering perspective, the system is modular, testable, and explainable. In addition, the report positions the work in dialogue with compiler construction, constraint-based generation, and carefully bounded search in modern sequence systems.

2 Introduction

Regular expressions occupy a special place in computer science because they are simultaneously mathematical objects, programming tools, and practical interfaces between human specification and machine behavior. At the formal level, they denote regular languages. At the systems level, they are compiled and executed by engines. At the educational level, they offer a compact and highly expressive medium for understanding symbolic pattern description.

The assignment for this laboratory work asks for more than a superficial use of regular expressions. The requirement is to dynamically interpret the given expressions and generate valid strings accordingly. This is a decisive distinction. A trivial solution would hardcode the output patterns for a specific variant and merely imitate valid samples. Such a solution could print plausible-looking words, but it would fail to solve the general problem. It would neither interpret the symbolic expression nor scale across all variants.

For this reason, the present work adopts a compiler-inspired approach. Each input expression is normalized, parsed, converted into an abstract syntax tree (AST), traversed to generate candidate strings, and validated against an independently emitted regular expression. This architecture makes the implementation universal across variants and provides a principled explanation for each stage of computation.

The work also aims to overachieve relative to the minimum assignment requirements. Beyond dynamic generation, it includes a trace mode that shows how a word was produced, a metric layer that estimates structural complexity, a visualization subsystem for ASTs

and traces, and a polished presentation/report package for oral defense. The result is not merely a script that works, but a small formal-language processing pipeline that can be explained, defended, and extended.

3 Assignment Interpretation and Objectives

The laboratory brief defines the topic as regular expressions and requires a codebase that can generate valid combinations of symbols according to several complex expressions shown in handwritten form. The core requirement is explicitly dynamic interpretation: the student must give a set of regular expressions as input and obtain valid words as output. The brief additionally states that expressions with unbounded repetition should be limited to at most five expansions, and it offers a bonus point for a function that shows the sequence of processing.

These requirements lead directly to the following engineering objectives:

1. Construct a universal representation for regular expressions that preserves the structure required for generation.
2. Parse handwritten-style notation reliably, including superscripts and optional whitespace.
3. Generate valid words without hardcoding expression-specific output templates.
4. Bound unbounded repetition operators in accordance with the assignment's safety rule.
5. Validate generated outputs independently so that the generator does not act as its own sole judge.
6. Expose the generation path through a readable step-by-step trace.
7. Cover all four variants in one architecture rather than building separate logic per variant.

The resulting system is therefore judged not only by whether it outputs correct words, but also by whether it embodies the correct computational idea. Universality, explainability, and verification are part of the deliverable.

4 Theoretical Background

4.1 Regular Expressions as Descriptions of Regular Languages

Let Σ be a finite alphabet. A regular expression over Σ denotes a set of strings, that is, a language $L \subseteq \Sigma^*$. The classical operators used in such descriptions include:

- **concatenation**, where AB denotes strings obtained by taking a string from A followed by a string from B ;

- **alternation**, where $A|B$ denotes the union of the two languages;
- **Kleene star**, where A^* denotes zero or more repetitions of strings from A ;
- **plus**, where A^+ denotes one or more repetitions of strings from A ;
- **optionality**, where $A?$ denotes either the empty string or one instance of A ;
- **exact or bounded powers**, such as A^n or $A\{m, n\}$.

While the formal semantics of these operators are well known, the assignment adds an operational rule: if repetition is unbounded, generation must still be finite. This introduces an implementation distinction between the meaning of the language and the strategy used to enumerate representatives from that language.

4.2 Why an AST is the Right Internal Form

Parsing expressions directly during generation would be both error-prone and difficult to explain. A more principled architecture separates syntax analysis from execution. The role of the AST is to provide a structural, typed representation of the expression. In this work, every parsed expression is transformed into a tree over four node classes:

- **Literal**,
- **Concat**,
- **Alternation**,
- **Repeat**.

This design is important for both correctness and pedagogy. Correctness improves because each operator is handled uniformly once it has been parsed into a typed node. Pedagogy improves because the AST makes it possible to show students and evaluators what structure the machine believes the expression has.

4.3 Bounded Generation as an Operational Strategy

The assignment's limit of five for unbounded repetition is best understood as a bounded generation policy, not a redefinition of regex semantics. The language of A^* remains infinite in theory. The engine, however, enumerates a finite subset of it for practical purposes. Formally, if a repeat node has no explicit upper bound, the implementation uses:

$$\text{upper}(A^*) = \text{max_repeat}, \quad \text{upper}(A^+) = \text{max_repeat}$$

with the default value `max_repeat = 5`. This turns the task into a finite combinatorial problem while preserving the intended acceptance logic.

5 System Architecture

The architecture of the system follows a clean, pipeline-oriented design. Each stage has one clear responsibility and communicates through well-defined data structures. This makes the code easier to test, easier to explain during presentation, and easier to extend later.

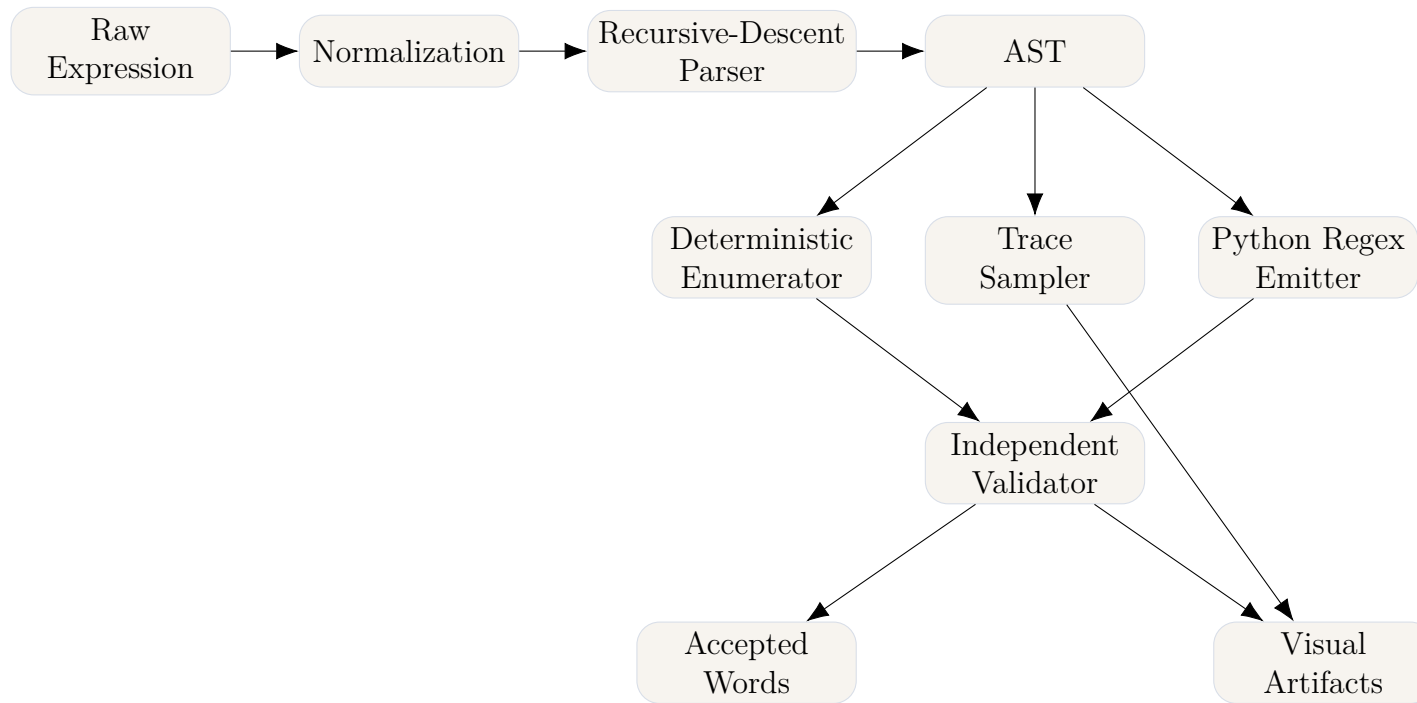


Figure 1: Processing pipeline of the Lab 4 regular-expression engine.

At the module level, the project is organized as follows:

- `src/parser.py`: text normalization and recursive-descent parsing,
- `src/ast_nodes.py`: typed AST node definitions,
- `src/generator.py`: deterministic and random generation logic,
- `src/visualization.py`: Mermaid-oriented diagram generation and structural metrics,
- `src/variants.py`: universal storage of all assignment variants,
- `main.py`: command-line orchestration.

The separation is deliberate. Parsing is not mixed with generation, generation is not mixed with validation, and the assignment data is not mixed with the operational engine.

6 Parsing and Normalization

6.1 Normalization Layer

The assignment images contain handwritten expressions, which introduces notation that is visually clear to a human but not necessarily explicit enough for a parser. In particular, superscripts such as ² and ⁵ may be represented using Unicode superscript characters, and spaces may appear inconsistently.

The normalization layer solves this by applying three simple but essential transformations:

1. remove whitespace,
2. convert Unicode superscripts to explicit caret notation,
3. preserve all other symbolic structure exactly.

This design is intentionally conservative. Normalization does not reinterpret the expression semantically; it only converts surface notation into a stable textual form that the parser can consume deterministically.

6.2 Recursive-Descent Strategy

The parser implements a recursive-descent strategy with explicit precedence handling. The precedence hierarchy is:

repetition > concatenation > alternation

This is implemented through a stack of parsing functions in which each layer delegates to the next tighter one. The parser begins with alternation, which internally calls concatenation, which in turn calls repetition, which finally calls atomic symbol parsing. This mirrors standard compiler design and keeps each function conceptually small.

Listing 1: Representative recursive-descent parsing logic for alternation and concatenation.

```
1 def _parse_union(self) -> RegexNode:
2     options = [self._parse_concat()]
3     while self._peek() == "|":
4         self._consume("|")
5         options.append(self._parse_concat())
6     if len(options) == 1:
7         return options[0]
8     return Alternation(tuple(options))
9
10 def _parse_concat(self) -> RegexNode:
11     parts = []
12     while True:
13         token = self._peek()
14         if token is None or token in ")|":
15             break
```



```

16         parts.append(self._parse_repetition())
17     if not parts:
18         return Literal("")
19     if len(parts) == 1:
20         return parts[0]
21     return Concat(tuple(parts))

```

The parser also supports:

- parenthesized groups,
- escaped literals,
- exact powers such as $\hat{5}$,
- braced quantifiers such as $\{m,n\}$,
- epsilon as the empty literal.

Invalid syntax is reported through a custom `RegexSyntaxError`. This is useful not only for debugging, but also for defending the implementation academically: malformed input is rejected explicitly rather than failing silently.

7 Generation Algorithms

7.1 Deterministic Enumeration

Deterministic generation is the engine's evidence-producing mode. Given an AST, it traverses the structure and enumerates valid words while enforcing two safety conditions:

- the number of results is limited by `max_results`,
- unbounded repetition uses the assignment-imposed cap `max_repeat = 5`.

The implementation is compositional. A literal yields one string. An alternation merges the languages of its options. A concatenation computes pairwise concatenations of partial outputs. A repeat node enumerates allowed counts and expands its child language accordingly.

Listing 2: Core idea of deterministic generation for repetition nodes.

```

1 if isinstance(node, Repeat):
2     upper = _upper_repeat_bound(node, max_repeat)
3     child_words = generate_language(
4         node.node, max_repeat=max_repeat, max_results=max_results
5     )
6     words = []
7     for count in range(node.min_times, upper + 1):
8         block = [""]

```

```

9         for _ in range(count):
10             block = _concat_words(block, child_words, max_results)
11             words = _merge_unique([*words, *block], max_results)
12     return words

```

This approach is conceptually straightforward, which makes it appropriate for a teaching-oriented lab. At the same time, the use of uniqueness merging and output limits prevents the algorithm from producing uncontrolled duplication or explosion during normal use.

7.2 Random Trace Sampling

The second generation mode creates one valid word together with a trace of how that word was obtained. This mode supports the bonus requirement from the brief: to show the sequence of processing steps.

Each trace step records three fields:

- a **path**, identifying the location in the AST,
- an **action**, such as choosing an alternative or expanding repetition,
- a **detail**, describing the concrete choice made.

This design converts the generator from a black box into an explainable symbolic system. For presentation purposes, it also creates a strong narrative bridge between formal grammars and practical tooling.

8 Independent Validation and Visualization

8.1 Validation Through an Independent Backend

One of the most important design decisions in this laboratory work is the separation between generation and validation. The generator produces candidate strings from the AST. The validator then converts the same AST into a Python-compatible regular expression and checks each candidate using `re.fullmatch`.

This matters because a generator can accidentally be self-consistent while still being wrong. By validating against an independently emitted formal pattern, the project gains a second source of truth.

Listing 3: Validation strategy for generated words.

```

1 ast = parse_regex(expression)
2 compiled = re.compile(f"^{to_python_regex(ast)}$")
3 invalid = [word for word in words if compiled.fullmatch(word) is
4             None]
5 if invalid:
6     raise RuntimeError("Generation mismatch detected.")

```

8.2 Visualization Layer

The implementation exports Mermaid diagrams for:

- AST structure,
- high-level processing pipeline,
- generation traces.

This is especially useful in an oral defense because it allows the invisible symbolic process to be turned into a visible artifact. The report itself uses $\text{\LaTeX}$ figures for publication quality, while the engine keeps Mermaid output for portability and fast iteration.

9 Universal Variant Coverage

The assignment is organized into four variants, each containing three regular expressions. A weak solution would focus only on one of them. The present implementation instead stores all four variants explicitly and interprets all twelve expressions through the same parser and generator.

9.1 Canonicalized Expressions

Table 1: Canonicalized machine-readable expressions for all assignment variants.

Var.	Expression 1	Expression 2	Expression 3
1	$(a b)(c d)E+G?$	$P(Q R S)T(UV W X)*Z+$	$1(0 1)*2(3 4)\hat{5}(3)(6)$
2	$M?N\hat{2}(O P)\hat{3}Q*R+$	$(X Y Z)\hat{3}(8)+(9 0)$	$(H I)(J K)L*N?$
3	$0(P Q R)+2(3 4)$	$A*B(C D E)F(G H I)\hat{2}$	$J+K(L M N)*0?(P Q)\hat{3}$
4	$(S T)(U V)W*Y+24$	$L(M N)0\hat{3}P*Q(2 3)$	$R*S(T U V)W(X Y Z)\hat{2}$

These expressions are not arbitrary inventions. They are explicit textual interpretations of the handwritten formulas in the assignment. When ambiguities appeared in adjacency or superscript placement, the expressions were grouped explicitly so that parser semantics remained faithful to the shown examples.

9.2 Variant Task Images

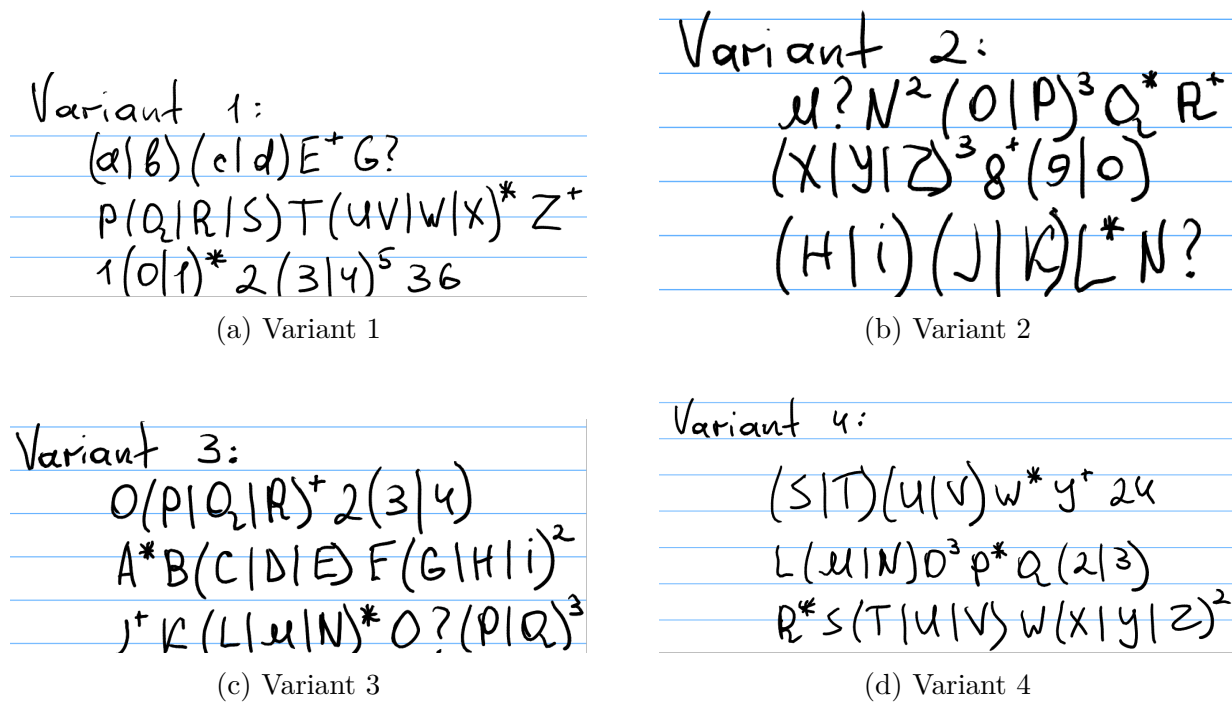


Figure 2: Official assignment variants used as the source specification for the universal generator.

9.3 Variant-by-Variant Discussion

Variant 1 combines front-loaded alternation with both optional and repeated suffix logic. It is a good early test of whether the parser correctly distinguishes between branching and sequential composition. Its third expression additionally verifies exact repetition through the power operator.

Variant 2 is notable for its use of exact multiplicities such as N^2 and $(O|P)^3$. This makes it comparatively clean from a structural perspective and provides a strong check that exact repetition semantics are implemented correctly.

Variant 3 contains the most demanding bounded search space. In particular, the expression $J^+K(L|M|N)^*O?(P|Q)^3$ combines mandatory repetition, optionality, alternation, and exact suffix repetition. This expression produces the highest rough path count in the structural analysis.

Variant 4 is especially useful in oral presentation because its outputs are visually easy to read. The expression $(S|T)(U|V)W^*Y^+24$ clearly displays prefix branching, bounded repetition, and a fixed numeric suffix, making it ideal as a pedagogical walkthrough case.

10 Empirical Analysis Across Variants

To move beyond anecdotal correctness, the implementation computes structural metrics for every expression. The following table summarizes the main results under the default

repeat cap.

Table 2: Structural metrics for all twelve expressions.

Expr.	Nodes	Depth	Alt.	Repeats	Rough paths
V1.E1	11	3	2	2	40
V1.E2	16	5	2	2	5460
V1.E3	13	4	2	2	2016
V2.E1	13	4	1	5	480
V2.E2	11	4	2	2	270
V2.E3	11	3	2	2	48
V3.E1	11	4	2	1	726
V3.E2	14	4	2	2	162
V3.E3	15	4	2	4	29120
V4.E1	13	3	2	2	120
V4.E2	13	3	2	2	24
V4.E3	14	4	2	2	162

The rough path count is not intended as an exact language cardinality. Rather, it is a bounded structural proxy that captures how rapidly the search space can grow when alternation and repetition interact. This proxy highlights the central engineering insight of the project: even in a classroom-sized regex engine, search behavior must be managed explicitly.

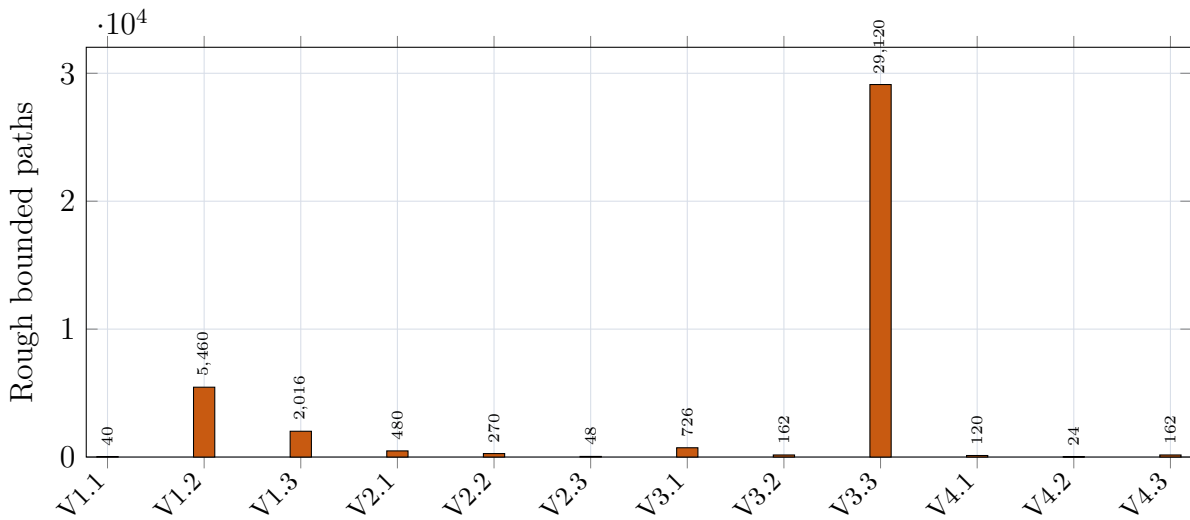


Figure 3: Rough bounded path counts across all expressions. Variant 3, Expression 3 is the dominant outlier.

11 Representative Generated Outputs

To demonstrate that the generator is not merely syntactically elegant but practically useful, Table 3 presents representative outputs obtained from the deterministic generator for each variant.

Table 3: Representative generated outputs from the bounded deterministic mode.

Variant	Representative outputs
1	acE, acEG, acEE, acEEG, acEEE, acEEEG; PQTZ, PQTZZ, PQTUVZ; 123333336, 123333436, 123334336
2	NN000R, NN000RR, NN000QR; XXX89, XXX80, XXX889; HJ, HJN, HJL, HJLLN
3	OP23, OQ24, OR24; BCFGG, BCFHH, BDFHI; JKPPP, JKPPQ, JKPQQ, JKQPQ
4	SUY24, SUYY24, SUWY24; LM000Q2, LM000PQ3, LN000PPQ2; STWXX, STWYY, RSTWYZ

These examples support two claims. First, the generator respects the symbolic structure of the expressions, including fixed suffixes and exact powers. Second, the output diversity changes naturally with structural complexity. Expressions with many branches and repeated regions produce visibly richer bounded languages.

12 Reproducible Evidence Bundle

To deepen the submission beyond static explanation, the project now includes an evidence-generation pipeline. A single CLI command can regenerate a structured analysis bundle containing:

- `summary.json`: machine-readable metrics, samples, traces, and benchmarks;
- `summary.md`: human-readable analytical summary;
- `terminal_transcript.txt`: a terminal-style walk-through across all variants.

The command used for the run documented in this report was:

Listing 4: Evidence bundle generation command used for this report.

```
1 python 4_regular_expressions/main.py --variant all --samples 4 --max-results 12 --validate
   --export-analysis-dir 4_regular_expressions/reports/evidence --benchmark-iterations 5
```

This addition strengthens the academic position of the report. Claims about complexity, speed, trace behavior, and validation are no longer only narrative; they are tied to generated artifacts inside the repository itself.

For oral defense and reproducibility, the evidence workflow can be described as a short four-stage protocol:

1. regenerate the analysis bundle from the CLI using the same repeat cap and benchmark settings;
2. inspect the produced JSON and markdown summaries for structural and timing data;
3. rerun the automated tests to check behavioral consistency;

4. rebuild the Sliddev deck and PDF report so that the presentation layer matches the measured evidence.

This protocol is intentionally simple. A reviewer does not need private tooling or hidden preprocessing scripts. The repository itself contains the commands and artifacts required to reproduce the main claims.

13 Benchmark Measurements

The evidence bundle records lightweight parse, generate, and validate timings for every expression. These measurements are not intended as universal performance claims, since they depend on machine and environment, but they are valuable for relative comparison within this project.

Table 4: Measured benchmark timings in milliseconds from the generated evidence bundle (`benchmark_iterations = 5`).

Expr.	Parse ms	Generate ms	Validate ms
V1.E1	0.0633	0.0526	0.0299
V1.E2	0.0537	0.0688	0.0277
V1.E3	0.0530	0.0795	0.0291
V2.E1	0.0510	0.1464	0.0331
V2.E2	0.0922	0.0972	0.0308
V2.E3	0.0456	0.0455	0.0342
V3.E1	0.0401	0.0450	0.0265
V3.E2	0.0583	0.0623	0.0338
V3.E3	0.0632	0.0785	0.0337
V4.E1	0.0528	0.1528	0.0279
V4.E2	0.0724	0.0607	0.0292
V4.E3	0.0620	0.0870	0.0323

Several observations emerge immediately:

1. Parsing remains uniformly inexpensive across expressions, staying well under 0.1 ms in the observed run.
2. Generation cost varies more strongly and reflects structural complexity more clearly than parse cost.
3. Variant 4, Expression 1 and Variant 2, Expression 1 are among the most expensive generation cases in this measurement set.
4. Validation is consistently inexpensive relative to generation, which supports the design choice of independent checking after enumeration.

These numbers support an important systems-level conclusion: the main engineering tension in this project is not syntax recognition, but safe traversal of bounded combinatorial structure.

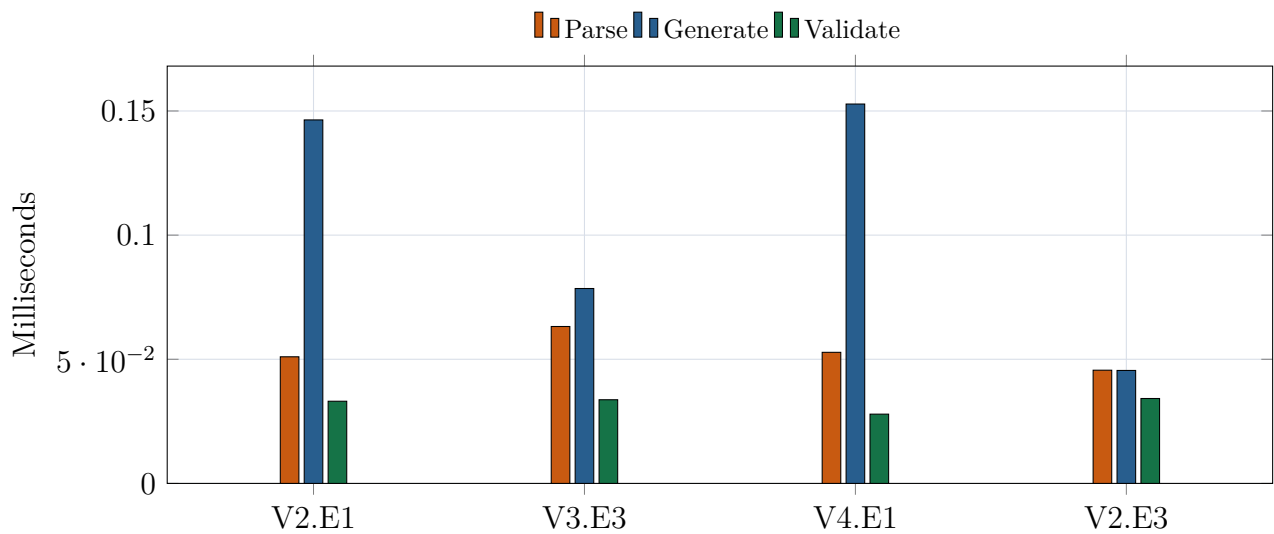


Figure 4: Representative stage timings. Generation varies far more than parsing or validation.

The grouped comparison confirms a qualitative pattern already visible in the table. Parsing remains consistently cheap. Validation also remains cheap. Generation is where structural differences surface operationally. This matters because it explains where engineering effort belongs: not in over-optimizing syntax reading, but in controlling traversal of repeated and branching subtrees.

14 Complexity, Safety, and Search Behavior

In the most general case, enumerating strings from a regex-like AST is inherently combinatorial. If a node contains many alternations and repeated substructures, the number of possible outputs grows rapidly. This is not a flaw in the implementation; it is a mathematical property of the language description itself.

Let:

- N denote the number of AST nodes,
- B denote the effective branching factor introduced by alternation,
- R denote the bounded repeat cap used for unbounded operators.

Then the number of bounded candidate strings can grow on the order of repeated products and sums involving B and R . This is why the system includes:

1. **repeat capping**, to avoid infinite expansion;

2. **max result limiting**, to avoid unbounded output volume;
3. **deduplication**, to suppress structural duplication in equivalent generation paths.

The project therefore treats complexity not as an afterthought but as part of the design. This is especially visible in Variant 3, Expression 3, where the structure would be extremely costly to enumerate naïvely.

15 Correctness Argument

The project is not a formal proof assistant, but it still benefits from an explicit correctness argument. In a laboratory context, this matters because evaluators need to know not only that outputs look plausible, but also why the pipeline should be trusted.

15.1 Claim 1: Structure Preservation During Parsing

After normalization, the parser processes the expression according to a fixed precedence order: repetition, concatenation, then alternation. Each grammar level has a dedicated parsing routine, and each routine constructs only the node types that correspond to that grammatical layer. Because of this design, the produced AST preserves the intended operator nesting of the normalized input.

Informally, the reason this works is classical recursive-descent structure: union parsing delegates to concatenation parsing, concatenation parsing delegates to repetition parsing, and repetition parsing delegates to atom parsing. Since tighter-binding operators are parsed first, the resulting tree respects precedence without requiring ad-hoc repair logic afterward.

15.2 Claim 2: Soundness of Bounded Generation

The deterministic generator traverses the AST compositionally. Literals emit fixed symbols, concatenation computes ordered products of child outputs, alternation computes unions of option outputs, and repetition expands a node across an allowed bounded interval. Therefore, every produced candidate is generated by a legal interpretation of the AST under the assignment's repeat cap.

This claim is intentionally phrased as *bounded soundness*. It does not say that the generator enumerates every string in an infinite language. Instead, it says that every emitted string belongs to the intended language once the assignment's finite-generation policy has been applied.

15.3 Claim 3: Independent Validation of Accepted Outputs

The generator is not allowed to be its own sole judge. For this reason, the project emits a Python-compatible validation target and checks candidates with full-match semantics. A string is therefore accepted only if it is both generated by the AST traversal and independently recognized by the validation backend.

This dual-check pattern is one of the strongest trust signals in the entire project. Even if a generation bug were introduced, the validator would still serve as an external acceptance filter. Conversely, if the emitted Python regex were malformed, the test suite and example matching checks would detect disagreement between the layers.

15.4 What This Correctness Story Does and Does Not Guarantee

The correctness argument supports the following conclusions:

- accepted outputs are structurally justified by the AST,
- accepted outputs are independently checked against regex semantics,
- bounded generation respects the assignment’s operational rule for infinite repetition.

It does not claim full equivalence with every industrial regex dialect, nor does it claim exhaustive enumeration of infinite languages. Those would be stronger goals than the assignment requires. For the present lab, the relevant result is that the system is sound, explainable, and consistent with the bounded-generation contract.

16 Verification and Testing

The repository includes automated tests in both `tests/test_lab4_regex.py` and `tests/test_lab4_and`. Together they cover the core engine and the evidence-generation layer. The tests cover the following dimensions:

- successful parsing of all expressions across all variants,
- consistency of official sample strings with interpreted expressions,
- agreement between generated outputs and Python regex validation,
- correct handling of bounded repetition,
- non-empty diagram and metric outputs,
- successful generation of step traces,
- successful export of the analysis bundle and transcript-oriented summaries.

This style of testing is pedagogically significant. It does not merely check one terminal command. Instead, it validates the entire pipeline from syntax analysis through semantic execution to visualization support.

Listing 5: Representative test excerpt.

```

1 def test_star_generation_respects_repeat_limit() -> None:
2     node = parse_regex("A*")
3     generated = generate_language(node, max_repeat=5, max_results
4         =20)
5     assert generated == ["", "A", "AA", "AAA", "AAAA", "AAAAA"]
6
7 def test_bonus_trace_returns_steps_and_valid_word() -> None:
8     expression = "J+K(L|M|N)*O?(P|Q)^3"
9     word, steps = explain_generation(expression, max_repeat=5,
10         seed=7)
11     assert word
12     assert steps

```

The emphasis on independent validation plus testing is one of the strongest aspects of the entire submission. It turns the implementation from a demonstration artifact into a defensible software system.

16.1 Terminal Transcript Evidence

The repository now includes a generated transcript in `reports/evidence/terminal_transcript.txt`. A representative excerpt is shown below:

Listing 6: Excerpt from the generated evidence transcript.

```

1 Regular Expressions Lab 4 - analysis transcript
2 seed=42, max_repeat=5, samples=4, benchmark_iterations=5
3
4 VARIANT 3
5 -----
6 Expression 3: J+K(L|M|N)*O?(P|Q)^3
7 Generated: {JKPPP, JKPPQ, JKPQP, JKPQQ}
8 Task examples: {JJKLOPPP, JKNQQQ}
9 Metrics: nodes=15, depth=4, alternations=2, repeats=4, rough_paths=29120
10 Benchmark: parse_ms=0.0632, generate_ms=0.0785, validate_ms=0.0337
11 Validation target: ^(?:J)+K(?:?:L|M|N))*(?:O)?(?:?:P|Q)){3}$
12 Trace sample: JKNPQP

```

This excerpt is useful because it combines the four most important proof dimensions in one place: sample outputs, assignment examples, structural metrics, and performance measurements. It demonstrates that the implementation can produce an integrated analytical view rather than isolated facts.

17 Threats to Validity and Reproducibility Considerations

Although the project is carefully engineered, several realism constraints still affect interpretation of the results.

17.1 Handwritten-to-Canonical Interpretation Risk

The source assignment variants are handwritten. Before any parser can operate, those formulas must be normalized into explicit machine-readable expressions. This introduces a small interpretation layer in which ambiguous notation must be resolved carefully. The project addresses this risk by documenting the canonicalized forms transparently and by presenting the original task images alongside those forms.

17.2 Environment-Dependent Benchmark Results

The benchmark values in this report are reproducible in methodology, but not guaranteed to be identical across machines. Millisecond-level timings depend on processor state, operating system behavior, and Python runtime conditions. For this reason, the benchmarks are used comparatively inside the project, not as universal performance claims.

17.3 Bounded Enumeration vs Full Language Coverage

The assignment explicitly authorizes a finite cap for otherwise unbounded repetition. This makes the generator operationally useful, but it also means the generated language is a bounded subset of the full theoretical language when `*` or `+` appear. The report therefore distinguishes clearly between regex semantics and the assignment-specific generation policy.

17.4 Repository-Level Reproducibility

One of the strongest aspects of the final submission is that its claims remain tied to concrete repository artifacts. The transcript, markdown summary, JSON evidence file, tests, Slidex deck, and PDF report are all generated from the same codebase. This alignment reduces the risk of a mismatch between implementation and presentation.

18 Discussion and ML/System Analogy

Although this project is fundamentally a formal-language assignment, it is useful to position it in dialogue with modern systems. The implemented architecture resembles a small symbolic decoding engine:

- the AST acts as a symbolic constraint program,
- alternation introduces branching possibilities,
- bounded repetition acts as a decoding-time regularizer,
- independent validation acts as a hard constraint checker.

This does *not* mean that the regex generator is equivalent to a neural sequence model. The analogy is more careful than that. It is intended to illuminate a shared systems idea:

generation under constraints. In both symbolic and modern ML systems, one often needs to generate outputs while guaranteeing structural correctness.

This analogy is pedagogically useful because it connects formal-language theory with a contemporary engineering intuition. The project remains entirely within the domain of symbolic parsing and generation, but it can still be discussed as a miniature version of constrained decoding logic.

19 Limitations and Future Work

No academic report is complete without a realistic account of current limitations. The present implementation intentionally prioritizes correctness, universality, and explainability over feature completeness. Its main limitations are:

- it targets the syntax required by the assignment rather than full PCRE-level regex semantics;
- it uses bounded generation for infinite operators rather than true language enumeration;
- its visualization output is Mermaid-oriented rather than based on a full AST graphics engine;
- ambiguity in handwritten notation still requires informed interpretation before canonicalization.

These limitations suggest several directions for future work:

1. add support for a richer regex syntax and more robust grouping rules,
2. compute formal automata from the expressions and compare generation with a separate automaton-based acceptance pipeline,
3. export diagrams directly to vector formats such as SVG or PDF for publication use,
4. add performance benchmarking for large bounded enumerations,
5. integrate an interactive browser dashboard for live classroom exploration.

Such extensions would not change the conceptual core of the project; rather, they would deepen its value as a general educational tool for regular-language processing.

20 Conclusion

This laboratory work set out to solve a problem that is easy to underestimate. At a glance, the assignment seems to ask for valid strings that match several regular expressions. In reality, however, it asks for a dynamic interpreter of symbolic specifications. The difference between those two tasks is the difference between imitation and computation.

The final implementation succeeds because it treats the expressions as structured programs. It normalizes handwritten notation, parses it into an AST, generates valid words through bounded traversal, validates those words independently, and exposes the internal reasoning path through traces and diagrams. Most importantly, it does so uniformly across all four variants.

From an academic perspective, the project demonstrates understanding of formal syntax, recursive-descent parsing, bounded search, and regular-language semantics. From a software-engineering perspective, it demonstrates modularity, verification, and explainability. From a presentation perspective, it provides a narrative strong enough to support oral defense and technical discussion.

The strongest claim that can be made about this lab is therefore simple: it is not merely a generator of strings, but a compact, defensible, and reusable regular-expression processing engine.

A Appendix A - Parser Source Listing

```
1 from __future__ import annotations
2
3 from dataclasses import dataclass
4
5 from .ast_nodes import Alternation, Concat, Literal, RegexNode,
    Repeat
6
7
8 class RegexSyntaxError(ValueError):
9     """Raised when an input regular expression cannot be parsed.
10     """
11
12 _SUPERScript_TO_DIGIT = {
13     "\u2070": "0",
14     "\u00b9": "1",
15     "\u00b2": "2",
16     "\u00b3": "3",
17     "\u2074": "4",
18     "\u2075": "5",
19     "\u2076": "6",
20     "\u2077": "7",
21     "\u2078": "8",
22     "\u2079": "9",
23 }
24
25
26 def normalize_expression(expression: str) -> str:
27     """Normalizes expression text so parser handles handwritten
28         notation.
29
30     - Removes whitespace.
```

```

30     - Converts Unicode superscripts to caret notation, e.g. A\
      u00b2 -> A^2.
31     """
32     output: list[str] = []
33     previous_was_superscript = False
34
35     for char in expression:
36         if char.isspace():
37             continue
38
39         if char in _SUPERSCRIPIT_TO_DIGIT:
40             if not previous_was_superscript:
41                 output.append("^")
42                 output.append(_SUPERSCRIPIT_TO_DIGIT[char])
43                 previous_was_superscript = True
44                 continue
45
46         previous_was_superscript = False
47         output.append(char)
48
49     return "".join(output)
50
51
52 @dataclass
53 class _RegexParser:
54     text: str
55     index: int = 0
56
57     def parse(self) -> RegexNode:
58         node = self._parse_union()
59         if self._peek() is not None:
60             raise RegexSyntaxError(
61                 f"Unexpected token '{self._peek()}' at index {self
62                     .index}."
63             )
64         return node
65
66     def _parse_union(self) -> RegexNode:
67         options = [self._parse_concat()]
68
69         while self._peek() == "|":
70             self._consume("|")
71             options.append(self._parse_concat())
72
73         if len(options) == 1:
74             return options[0]
75         return Alternation(tuple(options))
76
77     def _parse_concat(self) -> RegexNode:
78         parts: list[RegexNode] = []
79
80         while True:

```

```

80         token = self._peek()
81         if token is None or token in ")|":
82             break
83         parts.append(self._parse_repetition())
84
85     if not parts:
86         return Literal("")
87     if len(parts) == 1:
88         return parts[0]
89     return Concat(tuple(parts))
90
91 def _parse_repetition(self) -> RegexNode:
92     node = self._parse_atom()
93     seen_quantifier = False
94
95     while True:
96         token = self._peek()
97         if token not in {"*", "+", "?", "^", "{"}:
98             break
99         if seen_quantifier:
100             raise RegexSyntaxError(
101                 "Nested repetition operators are not supported
102                 in this lab syntax."
103             )
104
105         seen_quantifier = True
106
107         if token == "*":
108             self._consume("*")
109             node = Repeat(node=node, min_times=0, max_times=
110                 None)
111             continue
112
113         if token == "+":
114             self._consume("+")
115             node = Repeat(node=node, min_times=1, max_times=
116                 None)
117             continue
118
119         if token == "?":
120             self._consume("?")
121             node = Repeat(node=node, min_times=0, max_times=1)
122             continue
123
124         if token == "^":
125             self._consume("^")
126             amount = self._parse_number("Expected an integer
127                 after '^'.")
128             node = Repeat(node=node, min_times=amount,
129                 max_times=amount)
130             continue

```



```
127         min_times, max_times = self._parse_braced_quantifier()
128         node = Repeat(node=node, min_times=min_times,
129                        max_times=max_times)
130
131     return node
132
133 def _parse_braced_quantifier(self) -> tuple[int, int | None]:
134     self._consume("{")
135
136     if self._peek() == ",":
137         min_times = 0
138     else:
139         min_times = self._parse_number("Expected minimum
140                                     repetition in '{...}'")
141
142     if self._peek() == "}":
143         self._consume("}")
144         return min_times, min_times
145
146     self._consume(",")
147
148     if self._peek() == "}":
149         self._consume("}")
150         return min_times, None
151
152     max_times = self._parse_number("Expected maximum
153                                     repetition in '{m,n}'")
154     if max_times < min_times:
155         raise RegexSyntaxError("Quantifier maximum cannot be
156                                 smaller than minimum.")
157
158     self._consume("}")
159     return min_times, max_times
160
161 def _parse_atom(self) -> RegexNode:
162     token = self._peek()
163     if token is None:
164         raise RegexSyntaxError("Unexpected end of input while
165                                 parsing atom.")
166
167     if token == "(":
168         self._consume("(")
169         inner = self._parse_union()
170         self._consume(")")
171         return inner
172
173     if token == "\\":
174         self._consume("\\")
175         escaped = self._peek()
176         if escaped is None:
177             raise RegexSyntaxError("Dangling escape at end of
178                                     expression.")
```

```

173         self._consume(escaped)
174         return Literal(escaped)
175
176     if token == "\u03b5":
177         self._consume("\u03b5")
178         return Literal("")
179
180     if token in "+*?|)~{}":
181         raise RegexSyntaxError(f"Unexpected token '{token}' at
                                index {self.index}.")
182
183     self._consume(token)
184     return Literal(token)
185
186     def _parse_number(self, message: str) -> int:
187         start = self.index
188         while (current := self._peek()) is not None and current.
189             isdigit():
190             self.index += 1
191
192         if start == self.index:
193             raise RegexSyntaxError(message)
194
195         return int(self.text[start : self.index])
196
197     def _peek(self) -> str | None:
198         if self.index >= len(self.text):
199             return None
200         return self.text[self.index]
201
202     def _consume(self, expected: str) -> None:
203         current = self._peek()
204         if current != expected:
205             raise RegexSyntaxError(
206                 f"Expected '{expected}' at index {self.index},
207                 found '{current}'."
208             )
209         self.index += 1
210
211     def parse_regex(expression: str) -> RegexNode:
212         normalized = normalize_expression(expression)
213         if not normalized:
214             raise RegexSyntaxError("Expression is empty after
                                    normalization.")
215         parser = _RegexParser(normalized)
216         return parser.parse()

```

B Appendix B - Generator Source Listing

```
1 from __future__ import annotations
2
3 import random
4 import re
5 from dataclasses import dataclass
6 from typing import Iterable
7
8 from .ast_nodes import Alternation, Concat, Literal, RegexNode,
9     Repeat
10
11 from .parser import parse_regex
12
13 @dataclass(frozen=True, slots=True)
14 class GenerationStep:
15     path: str
16     action: str
17     detail: str
18
19 def _merge_unique(values: Iterable[str], limit: int) -> list[str]:
20     unique: list[str] = []
21     seen: set[str] = set()
22
23     for item in values:
24         if item in seen:
25             continue
26         seen.add(item)
27         unique.append(item)
28         if len(unique) >= limit:
29             break
30
31     return unique
32
33
34 def _concat_words(left: list[str], right: list[str], limit: int)
35     -> list[str]:
36     merged: list[str] = []
37     seen: set[str] = set()
38
39     for prefix in left:
40         for suffix in right:
41             candidate = prefix + suffix
42             if candidate in seen:
43                 continue
44             seen.add(candidate)
45             merged.append(candidate)
46             if len(merged) >= limit:
47                 return merged
48
49     return merged
```

```

50
51 def _upper_repeat_bound(node: Repeat, max_repeat: int) -> int:
52     if node.max_times is None:
53         return max_repeat
54     return node.max_times
55
56
57 def generate_language(
58     node: RegexNode,
59     *,
60     max_repeat: int = 5,
61     max_results: int = 200,
62 ) -> list[str]:
63     """Enumerates accepted words up to configurable limits.
64
65     max_repeat is applied only to unbounded repetition operators
66     (* and +).
67
68     """
69
70     if max_results <= 0:
71         return []
72
73     if isinstance(node, Literal):
74         return [node.value]
75
76     if isinstance(node, Alternation):
77         words: list[str] = []
78         for option in node.options:
79             option_words = generate_language(
80                 option, max_repeat=max_repeat, max_results=
81                 max_results
82             )
83             words = _merge_unique([*words, *option_words],
84                                   max_results)
85             if len(words) >= max_results:
86                 break
87         return words
88
89     if isinstance(node, Concat):
90         words = [""]
91         for part in node.parts:
92             part_words = generate_language(part, max_repeat=
93                 max_repeat, max_results=max_results)
94             words = _concat_words(words, part_words, max_results)
95             if not words:
96                 break
97         return words
98
99     if isinstance(node, Repeat):
100         upper = _upper_repeat_bound(node, max_repeat)
101         if upper < node.min_times:
102             return []

```

```

98
99     child_words = generate_language(
100         node.node, max_repeat=max_repeat, max_results=
            max_results
101     )
102     words: list[str] = []
103
104     for count in range(node.min_times, upper + 1):
105         block = [""]
106         for _ in range(count):
107             block = _concat_words(block, child_words,
                max_results)
108             if not block:
109                 break
110             words = _merge_unique([*words, *block], max_results)
111             if len(words) >= max_results:
112                 break
113
114     return words
115
116     raise TypeError(f"Unsupported node type: {type(node)!r}")
117
118
119 def generate_random_word(
120     node: RegexNode,
121     *,
122     max_repeat: int = 5,
123     rng: random.Random | None = None,
124     with_trace: bool = False,
125 ) -> tuple[str, list[GenerationStep]]:
126     """Generates one valid word and optionally returns expansion
        trace steps."""
127
128     random_source = rng or random.Random()
129     steps: list[GenerationStep] = []
130
131     def sample(current: RegexNode, path: str) -> str:
132         if isinstance(current, Literal):
133             if with_trace:
134                 steps.append(GenerationStep(path, "literal", repr(
                    current.value)))
135             return current.value
136
137         if isinstance(current, Alternation):
138             index = random_source.randrange(len(current.options))
139             if with_trace:
140                 steps.append(
141                     GenerationStep(
142                         path,
143                         "choose-alternative",
144                         f"option {index + 1} of {len(current.
                            options)}",

```

```

145         )
146     )
147     return sample(current.options[index], f"{path}|{index}"
148         ")
149
150     if isinstance(current, Concat):
151         if with_trace:
152             steps.append(
153                 GenerationStep(
154                     path,
155                     "concat",
156                     f"{len(current.parts)} parts",
157                 )
158             )
159         return "".join(sample(part, f"{path}.{i}") for i, part
160             in enumerate(current.parts))
161
162     if isinstance(current, Repeat):
163         upper = _upper_repeat_bound(current, max_repeat)
164         if upper < current.min_times:
165             raise ValueError("Invalid repetition limits for
166                 random generation.")
167
168         times = random_source.randint(current.min_times, upper
169             )
170         if with_trace:
171             steps.append(
172                 GenerationStep(
173                     path,
174                     "repeat",
175                     f"{times} times (range {current.min_times
176                         }..{upper})",
177                 )
178             )
179         return "".join(sample(current.node, f"{path}#{i}") for
180             i in range(times))
181
182     raise TypeError(f"Unsupported node type: {type(current)!r}"
183         ")
184
185     word = sample(node, "root")
186     return word, steps
187
188 def to_python_regex(node: RegexNode) -> str:
189     """Converts internal AST to a Python-compatible regex
190     expression."""
191
192     def emit(current: RegexNode) -> str:
193         if isinstance(current, Literal):
194             return re.escape(current.value)

```

```

189         if isinstance(current, Concat):
190             return "".join(emit(part) for part in current.parts)
191
192         if isinstance(current, Alternation):
193             content = "|".join(emit(option) for option in current.
194                               options)
195             return f"({content})"
196
197         if isinstance(current, Repeat):
198             inner = emit(current.node)
199             wrapped = f"({inner})"
200             if current.min_times == 0 and current.max_times is
201                 None:
202                 return f"{wrapped}*"
203             if current.min_times == 1 and current.max_times is
204                 None:
205                 return f"{wrapped}+"
206             if current.min_times == 0 and current.max_times == 1:
207                 return f"{wrapped}?"
208             if current.max_times is None:
209                 return f"{wrapped}{{{current.min_times},}}"
210             if current.min_times == current.max_times:
211                 return f"{wrapped}{{{current.min_times}}}"
212             return f"{wrapped}{{{current.min_times},{current.
213                               max_times}}}"
214
215         raise TypeError(f"Unsupported node type: {type(current)!r}")
216
217     return emit(node)
218
219 def explain_generation(
220     expression: str,
221     *,
222     max_repeat: int = 5,
223     seed: int = 42,
224 ) -> tuple[str, list[GenerationStep]]:
225     """Parses an expression and returns one generated word plus
226         full expansion trace."""
227
228     ast = parse_regex(expression)
229     rng = random.Random(seed)
230     return generate_random_word(ast, max_repeat=max_repeat, rng=
231                               rng, with_trace=True)

```

C Appendix C - Visualization Source Listing

```

1 from __future__ import annotations
2

```

```

3 from typing import Any
4
5 from .ast_nodes import Alternation, Concat, Literal, RegexNode,
  Repeat
6 from .generator import GenerationStep
7
8
9 def _escape_mermaid_label(text: str) -> str:
10     cleaned = text.replace("'", "'")
11     return cleaned.replace("\n", " ").strip()
12
13
14 def _node_kind(node: RegexNode) -> str:
15     if isinstance(node, Literal):
16         return "literal"
17     if isinstance(node, Alternation):
18         return "alternation"
19     if isinstance(node, Concat):
20         return "concat"
21     if isinstance(node, Repeat):
22         return "repeat"
23     return "unknown"
24
25
26 def _node_label(node: RegexNode) -> str:
27     if isinstance(node, Literal):
28         if node.value == "":
29             return "Literal epsilon"
30             return f"Literal {node.value}"
31
32     if isinstance(node, Alternation):
33         return f"Alternation {len(node.options)} options"
34
35     if isinstance(node, Concat):
36         return f"Concatenation {len(node.parts)} parts"
37
38     if isinstance(node, Repeat):
39         upper = "inf" if node.max_times is None else str(node.
40             max_times)
41         return f"Repeat {node.min_times}..{upper}"
42
43     return "Node"
44
45 def ast_to_mermaid(node: RegexNode, *, title: str | None = None)
46     -> str:
47     """Builds a Mermaid flowchart representation of a regex AST.
48         """
49
50     lines: list[str] = []
51     if title:

```



```

50     lines.extend(["---", f'title: "{_escape_mermaid_label(
51         title)}"', "---"])
52
53     lines.extend(
54         [
55             "flowchart TD",
56             "    classDef literal fill:#edf2ff,stroke:#3b5bdb,stroke-width:1px;",
57             "    classDef concat fill:#f3faf7,stroke:#2f9e44,stroke-width:1px;",
58             "    classDef alternation fill:#fff9db,stroke:#f08c00,stroke-width:1px;",
59             "    classDef repeat fill:#fff0f6,stroke:#c2255c,stroke-width:1px;",
60         ]
61     )
62
63     class_members: dict[str, list[str]] = {
64         "literal": [],
65         "concat": [],
66         "alternation": [],
67         "repeat": [],
68     }
69
70     next_id = 0
71
72     def visit(current: RegexNode) -> str:
73         nonlocal next_id
74         node_id = f"N{next_id}"
75         next_id += 1
76
77         kind = _node_kind(current)
78         label = _escape_mermaid_label(_node_label(current))
79         lines.append(f'    {node_id}["{label}"]')
80         if kind in class_members:
81             class_members[kind].append(node_id)
82
83         if isinstance(current, Concat):
84             for idx, part in enumerate(current.parts, start=1):
85                 child = visit(part)
86                 lines.append(f'    {node_id} -->|part {idx}| {child}')
87
88         elif isinstance(current, Alternation):
89             for idx, option in enumerate(current.options, start=1):
90                 child = visit(option)
91                 lines.append(f'    {node_id} -->|option {idx}| {child}')
92
93         elif isinstance(current, Repeat):
94             child = visit(current.node)

```

```

94         upper = "inf" if current.max_times is None else str(
95             current.max_times)
96         lines.append(f"        {node_id} -->|repeat {current.
97             min_times}..{upper}| {child}")
98
99         return node_id
100
101     root = visit(node)
102     lines.append(f"    ROOT([Regex Root]) --> {root}")
103
104     for class_name, members in class_members.items():
105         if members:
106             lines.append(f"        class {'','.join(members)} {
107                 class_name};")
108
109     return "\n".join(lines) + "\n"
110
111 def regex_pipeline_mermaid() -> str:
112     """High-level processing pipeline diagram for the lab workflow
113     ."""
114
115     return "\n".join(
116         [
117             "flowchart LR",
118             '    A[Input Regex] --> B[Normalization]',
119             '    B --> C[Recursive-Descent Parser]',
120             '    C --> D[AST Program]',
121             '    D --> E[Bounded Generator]',
122             '    E --> F[Candidate Words]',
123             '    D --> G[Python Regex Compiler]',
124             '    F --> H[Full-Match Validator]',
125             '    G --> H',
126             '    H --> I[Accepted Language Sample]',
127             '    D --> J[Trace / Explainability]',
128             '    J --> K[Mission Control Report]',
129         ]
130     ) + "\n"
131
132 def generation_trace_to_mermaid(word: str, steps: list[
133     GenerationStep]) -> str:
134     """Converts generation steps into a linear Mermaid flowchart.
135     """
136
137     lines = ["flowchart LR", '    S([Start]) --> T0["seeded decode
138         "]']
139     previous = "T0"
140
141     for i, step in enumerate(steps, start=1):
142         step_id = f"T{i}"
143         detail = _escape_mermaid_label(step.detail)

```

```

139     path = _escape_mermaid_label(step.path)
140     label = _escape_mermaid_label(f"{i:02d} {step.action} | {
        path} | {detail}")
141     lines.append(f'        {step_id}["{label}"]')
142     lines.append(f"        {previous} --> {step_id}")
143     previous = step_id
144
145     final_label = _escape_mermaid_label(f"output: {word}")
146     lines.append(f'        0(["{final_label}"])'')
147     lines.append(f"        {previous} --> 0")
148     return "\n".join(lines) + "\n"
149
150
151 def ast_metrics(node: RegexNode, *, max_repeat: int = 5) -> dict[
    str, Any]:
152     """Computes structural metrics used in deeper explanatory
        reports."""
153
154     def walk(current: RegexNode) -> tuple[int, int, int, int, int,
        int]:
155         # Returns:
156         # nodes, depth, alternations, repeats, literals,
        rough_paths
157         if isinstance(current, Literal):
158             return 1, 1, 0, 0, 1, 1
159
160         if isinstance(current, Concat):
161             child_stats = [walk(part) for part in current.parts]
162             nodes = 1 + sum(s[0] for s in child_stats)
163             depth = 1 + max((s[1] for s in child_stats), default
                =0)
164             alternations = sum(s[2] for s in child_stats)
165             repeats = sum(s[3] for s in child_stats)
166             literals = sum(s[4] for s in child_stats)
167
168             rough_paths = 1
169             for stats in child_stats:
170                 rough_paths *= max(stats[5], 1)
171             rough_paths = min(rough_paths, 1_000_000)
172
173             return nodes, depth, alternations, repeats, literals,
                rough_paths
174
175         if isinstance(current, Alternation):
176             child_stats = [walk(option) for option in current.
                options]
177             nodes = 1 + sum(s[0] for s in child_stats)
178             depth = 1 + max((s[1] for s in child_stats), default
                =0)
179             alternations = 1 + sum(s[2] for s in child_stats)
180             repeats = sum(s[3] for s in child_stats)
181             literals = sum(s[4] for s in child_stats)

```

```

182         rough_paths = min(sum(s[5] for s in child_stats), 1
183                             _000_000)
184     return nodes, depth, alternations, repeats, literals,
185           rough_paths
186
187     if isinstance(current, Repeat):
188         inner = walk(current.node)
189         upper = max_repeat if current.max_times is None else
190             current.max_times
191         upper = max(upper, current.min_times)
192
193         rough_paths = 0
194         for k in range(current.min_times, upper + 1):
195             rough_paths += min(inner[5] ** k, 1_000_000)
196             rough_paths = min(rough_paths, 1_000_000)
197
198         return (
199             1 + inner[0],
200             1 + inner[1],
201             inner[2],
202             1 + inner[3],
203             inner[4],
204             rough_paths,
205         )
206
207     return 1, 1, 0, 0, 0, 1
208
209 nodes, depth, alternations, repeats, literals, rough_paths =
210 walk(node)
211 return {
212     "nodes": nodes,
213     "depth": depth,
214     "alternations": alternations,
215     "repeats": repeats,
216     "literals": literals,
217     "rough_paths": rough_paths,
218 }

```

D Appendix D - Test Suite Listing

```

1 from __future__ import annotations
2
3 import re
4
5 from src.generator import explain_generation, generate_language,
6   to_python_regex
7 from src.parser import normalize_expression, parse_regex
8 from src.variants import TASK_EXAMPLES, VARIANTS
9 from src.visualization import ast_metrics, ast_to_mermaid,
10   generation_trace_to_mermaid

```

```
9
10
11 def _compiled(expr: str) -> re.Pattern[str]:
12     node = parse_regex(expr)
13     return re.compile(f"^{to_python_regex(node)}$")
14
15
16 def test_all_variant_patterns_parse() -> None:
17     for variant in VARIANTS.values():
18         for expression in variant.expressions:
19             parse_regex(expression)
20
21
22 def test_given_task_examples_match_interpreted_variants() -> None:
23     for variant_id, variant in VARIANTS.items():
24         examples_for_variant = TASK_EXAMPLES[variant_id]
25         for expression, examples in zip(variant.expressions,
26                                         examples_for_variant):
27             pattern = _compiled(expression)
28             for sample in examples:
29                 assert pattern.fullmatch(sample) is not None
30
31
32 def test_generated_words_match_python_regex_engine() -> None:
33     for variant in VARIANTS.values():
34         for expression in variant.expressions:
35             node = parse_regex(expression)
36             pattern = _compiled(expression)
37             generated = generate_language(node, max_repeat=5,
38                                         max_results=30)
39             assert generated
40             for word in generated:
41                 assert pattern.fullmatch(word) is not None
42
43
44 def test_star_generation_respects_repeat_limit() -> None:
45     node = parse_regex("A*")
46     generated = generate_language(node, max_repeat=5, max_results
47                                 =20)
48     assert generated == ["", "A", "AA", "AAA", "AAAA", "AAAAA"]
49
50
51 def test_bonus_trace_returns_steps_and_valid_word() -> None:
52     expression = "J+K(L|M|N)*O?(P|Q)^3"
53     word, steps = explain_generation(expression, max_repeat=5,
54                                     seed=7)
55     assert word
56     assert steps
57
58     pattern = _compiled(expression)
59     assert pattern.fullmatch(word) is not None
```

```

57
58 def test_unicode_superscript_normalization() -> None:
59     expression = "(A|B)" + chr(0x00B2) + "C" + chr(0x2075)
60     assert normalize_expression(expression) == "(A|B)^2C^5"
61
62
63 def test_visualization_mermaid_outputs_have_expected_sections() ->
    None:
64     node = parse_regex("A(B|C)+D?")
65     diagram = ast_to_mermaid(node, title="demo")
66     assert "flowchart TD" in diagram
67     assert "Regex Root" in diagram
68     assert "Alternation" in diagram
69
70
71 def test_trace_mermaid_and_metrics_are_non_empty() -> None:
72     expression = "J+K(L|M|N)*O?(P|Q)^3"
73     word, steps = explain_generation(expression, max_repeat=5,
74                                     seed=9)
75     trace_diagram = generation_trace_to_mermaid(word, steps)
76     assert "flowchart LR" in trace_diagram
77     assert "output:" in trace_diagram
78
79     metrics = ast_metrics(parse_regex(expression), max_repeat=5)
80     assert metrics["nodes"] > 0
81     assert metrics["depth"] > 0
82     assert metrics["rough_paths"] > 0

```

E Appendix E - Generated Terminal Transcript

```

1 Regular Expressions Lab 4 - analysis transcript
2 seed=42, max_repeat=5, samples=4, benchmark_iterations=5
3
4 VARIANT 1
5 -----
6 Expression 1: (a|b)(c|d)E+G?
7 Generated: {acE, acEG, acEE, acEEG}
8 Task examples: {acEG, bdE, adEEG}
9 Metrics: nodes=11, depth=3, alternations=2, repeats=2, rough_paths=40
10 Benchmark: parse_ms=0.028, generate_ms=0.0305, validate_ms=0.0224
11 Validation target: ^(?:a|b)(?:c|d)(?:E)+(?:G)?$
12 Trace sample: acEEE
13   - path=root action=concat detail=4 parts
14   - path=root.0 action=choose-alternative detail=option 1 of 2
15   - path=root.0|0 action=literal detail='a'
16   - path=root.1 action=choose-alternative detail=option 1 of 2
17   - path=root.1|0 action=literal detail='c'
18
19 Expression 2: P(Q|R|S)T(UV|W|X)*Z+
20 Generated: {PQTZ, PQTZZ, PQTZZZ, PQTZZZZ}
21 Task examples: {PQTUVUVZ, PRTWWWWZ}
22 Metrics: nodes=16, depth=5, alternations=2, repeats=2, rough_paths=5460
23 Benchmark: parse_ms=0.053, generate_ms=0.0506, validate_ms=0.0196
24 Validation target: ^P(?:Q|R|S)T(?:UV|W|X)*(?:Z)+$
25 Trace sample: PSTZZZZZ
26   - path=root action=concat detail=5 parts
27   - path=root.0 action=literal detail='P'
28   - path=root.1 action=choose-alternative detail=option 3 of 3

```

```

29 - path=root.1|2 action=literal detail='S'
30 - path=root.2 action=literal detail='T'
31
32 Expression 3: 1(0|1)*2(3|4)^5(3)(6)
33 Generated: {123333336, 123333436, 123334336, 123334436}
34 Task examples: {1023333336, 1124444436}
35 Metrics: nodes=13, depth=4, alternations=2, repeats=2, rough_paths=2016
36 Benchmark: parse_ms=0.0805, generate_ms=0.0706, validate_ms=0.0289
37 Validation target: ^1(?:0|1)*2(?:3|4){5}36$
38 Trace sample: 1101124433436
39 - path=root action=concat detail=6 parts
40 - path=root.0 action=literal detail='1'
41 - path=root.1 action=repeat detail=4 times (range 0..5)
42 - path=root.1#0 action=choose-alternative detail=option 2 of 2
43 - path=root.1#0|1 action=literal detail='1'
44
45 VARIANT 2
46 -----
47 Expression 1: M?N^2(0|P)^3Q*R+
48 Generated: {NNOOOR, NNOOORR, NNOOORRR, NNOOORRRR}
49 Task examples: {MNNOOQR, NNPPPPQRRR}
50 Metrics: nodes=13, depth=4, alternations=1, repeats=5, rough_paths=480
51 Benchmark: parse_ms=0.0346, generate_ms=0.0684, validate_ms=0.0319
52 Validation target: ^(?:M)?(?:N){2}(?:0|P){3}(?:Q)*(?:R)+$
53 Trace sample: NNPOQQR
54 - path=root action=concat detail=5 parts
55 - path=root.0 action=repeat detail=0 times (range 0..1)
56 - path=root.1 action=repeat detail=2 times (range 2..2)
57 - path=root.1#0 action=literal detail='N'
58 - path=root.1#1 action=literal detail='N'
59
60 Expression 2: (X|Y|Z)^3(8)+(9|0)
61 Generated: {XXX89, XXX80, XXX889, XXX880}
62 Task examples: {XXX89, YYY88889}
63 Metrics: nodes=11, depth=4, alternations=2, repeats=2, rough_paths=270
64 Benchmark: parse_ms=0.0369, generate_ms=0.0392, validate_ms=0.0135
65 Validation target: ^(?:X|Y|Z){3}(?:8)+(?:9|0)$
66 Trace sample: YZZ89
67 - path=root action=concat detail=3 parts
68 - path=root.0 action=repeat detail=3 times (range 3..3)
69 - path=root.0#0 action=choose-alternative detail=option 2 of 3
70 - path=root.0#0|1 action=literal detail='Y'
71 - path=root.0#1 action=choose-alternative detail=option 3 of 3
72
73 Expression 3: (H|I)(J|K)L*N?
74 Generated: {HJ, HJN, HJL, HJLN}
75 Task examples: {HJLLN, IKLLLLLL}
76 Metrics: nodes=11, depth=3, alternations=2, repeats=2, rough_paths=48
77 Benchmark: parse_ms=0.027, generate_ms=0.0374, validate_ms=0.0161
78 Validation target: ^(?:H|I)(?:J|K)(?:L)*(?:N)?$
79 Trace sample: IJLLLN
80 - path=root action=concat detail=4 parts
81 - path=root.0 action=choose-alternative detail=option 2 of 2
82 - path=root.0|1 action=literal detail='I'
83 - path=root.1 action=choose-alternative detail=option 1 of 2
84 - path=root.1|0 action=literal detail='J'
85
86 VARIANT 3
87 -----
88 Expression 1: 0(P|Q|R)+2(3|4)
89 Generated: {OP23, OP24, OQ23, OQ24}
90 Task examples: {OPP23, OQQQQ24}
91 Metrics: nodes=11, depth=4, alternations=2, repeats=1, rough_paths=726
92 Benchmark: parse_ms=0.0347, generate_ms=0.0276, validate_ms=0.0136
93 Validation target: ^0(?:P|Q|R){2}(?:3|4)$
94 Trace sample: ORPRR23
95 - path=root action=concat detail=4 parts
96 - path=root.0 action=literal detail='0'
97 - path=root.1 action=repeat detail=4 times (range 1..5)
98 - path=root.1#0 action=choose-alternative detail=option 3 of 3
99 - path=root.1#0|2 action=literal detail='R'
100
101 Expression 2: A*B(C|D|E)F(G|H|I)^2
102 Generated: {BCFGG, BCFGH, BCFGI, BCFHG}

```

```

103 Task examples: {AAABCFGG, AAAAAABDFHH}
104 Metrics: nodes=14, depth=4, alternations=2, repeats=2, rough_paths=162
105 Benchmark: parse_ms=0.0355, generate_ms=0.0419, validate_ms=0.0113
106 Validation target: ^(?:A)*B(?:C|D|E)F(?:G|H|I){2}$
107 Trace sample: AAAABDFGG
108   - path=root action=concat detail=5 parts
109   - path=root.0 action=repeat detail=4 times (range 0..5)
110   - path=root.0#0 action=literal detail='A'
111   - path=root.0#1 action=literal detail='A'
112   - path=root.0#2 action=literal detail='A'
113
114 Expression 3: J+K(L|M|N)*O?(P|Q)^3
115 Generated: {JKPPP, JKPPQ, JKPQP, JKPQQ}
116 Task examples: {JJKLOPPP, JKNQQQ}
117 Metrics: nodes=15, depth=4, alternations=2, repeats=4, rough_paths=29120
118 Benchmark: parse_ms=0.0407, generate_ms=0.0597, validate_ms=0.0154
119 Validation target: ^(?:J)+K(?:L|M|N)*(?:O)?(?:P|Q){3}$
120 Trace sample: JKNPQP
121   - path=root action=concat detail=5 parts
122   - path=root.0 action=repeat detail=1 times (range 1..5)
123   - path=root.0#0 action=literal detail='J'
124   - path=root.1 action=literal detail='K'
125   - path=root.2 action=repeat detail=1 times (range 0..5)
126
127 VARIANT 4
128 -----
129 Expression 1: (S|T)(U|V)W*Y+24
130 Generated: {SUY24, SUYY24, SUYYYY24, SUYYYYY24}
131 Task examples: {SUWWY24, SVWY24}
132 Metrics: nodes=13, depth=3, alternations=2, repeats=2, rough_paths=120
133 Benchmark: parse_ms=0.0385, generate_ms=0.0676, validate_ms=0.0482
134 Validation target: ^(?:S|T)(?:U|V)(?:W)*(?:Y)+24$
135 Trace sample: SUWWYYY24
136   - path=root action=concat detail=6 parts
137   - path=root.0 action=choose-alternative detail=option 1 of 2
138   - path=root.0|0 action=literal detail='S'
139   - path=root.1 action=choose-alternative detail=option 1 of 2
140   - path=root.1|0 action=literal detail='U'
141
142 Expression 2: L(M|N)O^3P*Q(2|3)
143 Generated: {LM000Q2, LM000Q3, LM000PQ2, LM000PQ3}
144 Task examples: {LM000PPPPQ2, LN000PQ3}
145 Metrics: nodes=13, depth=3, alternations=2, repeats=2, rough_paths=24
146 Benchmark: parse_ms=0.0415, generate_ms=0.0547, validate_ms=0.0299
147 Validation target: ^L(?:M|N)(?:O){3}(?:P)*Q(?:2|3)$
148 Trace sample: LM000PPPPQ2
149   - path=root action=concat detail=6 parts
150   - path=root.0 action=literal detail='L'
151   - path=root.1 action=choose-alternative detail=option 1 of 2
152   - path=root.1|0 action=literal detail='M'
153   - path=root.2 action=repeat detail=3 times (range 3..3)
154
155 Expression 3: R*S(T|U|V)W(X|Y|Z)^2
156 Generated: {STWXX, STWXY, STWXZ, STWYX}
157 Task examples: {RSTWXX, RRSUWYYY}
158 Metrics: nodes=14, depth=4, alternations=2, repeats=2, rough_paths=162
159 Benchmark: parse_ms=0.1118, generate_ms=0.082, validate_ms=0.0306
160 Validation target: ^(?:R)*S(?:T|U|V)W(?:X|Y|Z){2}$
161 Trace sample: RRSUWYY
162   - path=root action=concat detail=5 parts
163   - path=root.0 action=repeat detail=2 times (range 0..5)
164   - path=root.0#0 action=literal detail='R'
165   - path=root.0#1 action=literal detail='R'
166   - path=root.1 action=literal detail='S'

```

F Appendix F - Evidence Summary

```

1 # Lab 4 Analysis Bundle
2

```



```

3  ## Meta
4
5  - Variants covered: 4
6  - Expressions covered: 12
7  - Repeat cap: 5
8  - Sample count per expression: 4
9  - Benchmark iterations: 5
10 - Heaviest bounded expression: variant 3 / expression 3 (29120 rough paths)
11
12 ## Variant 1
13
14 ### Expression 1
15
16 - Regex: '(a|b)(c|d)E+G?'
17 - Validation target: '^(?:a|b)(?:c|d)(?:E)+(?:G)?$'
18 - Metrics: nodes=11, depth=3, alternations=2, repeats=2, rough_paths=40
19 - Benchmark: parse_ms=0.028, generate_ms=0.0305, validate_ms=0.0224
20 - Generated samples: 'acE, acEG, acEE, acEEG'
21 - Official examples: 'acEG, bdE, adEEG'
22 - Trace sample: 'acEEE'
23
24 ### Expression 2
25
26 - Regex: 'P(Q|R|S)T(UV|W|X)*Z+'
27 - Validation target: '^P(?:Q|R|S)T(?:UV|W|X)*(?:Z)+$'
28 - Metrics: nodes=16, depth=5, alternations=2, repeats=2, rough_paths=5460
29 - Benchmark: parse_ms=0.053, generate_ms=0.0506, validate_ms=0.0196
30 - Generated samples: 'PQTZ, PQTZZ, PQTZZZ, PQTZZZZ'
31 - Official examples: 'PQTUVUVZ, PRTWWWWZ'
32 - Trace sample: 'PSTZZZZZ'
33
34 ### Expression 3
35
36 - Regex: '1(0|1)*2(3|4)^5(3)(6)'
37 - Validation target: '^1(?:0|1)*2(?:3|4){5}36$'
38 - Metrics: nodes=13, depth=4, alternations=2, repeats=2, rough_paths=2016
39 - Benchmark: parse_ms=0.0805, generate_ms=0.0706, validate_ms=0.0289
40 - Generated samples: '123333336, 123333436, 123334336, 123334436'
41 - Official examples: '1023333336, 1124444436'
42 - Trace sample: '1101124433436'
43
44 ## Variant 2
45
46 ### Expression 1
47
48 - Regex: 'M?N^2(O|P)^3Q*R+'
49 - Validation target: '^(?:M)?(?:N){2}(?:O|P){3}(?:Q)*(?:R)+$'
50 - Metrics: nodes=13, depth=4, alternations=1, repeats=5, rough_paths=480
51 - Benchmark: parse_ms=0.0346, generate_ms=0.0684, validate_ms=0.0319
52 - Generated samples: 'NNOOOR, NNOOORR, NNOOORRR, NNOOORRRR'
53 - Official examples: 'MNNOOOQR, NNPPPPQQRRR'
54 - Trace sample: 'NNOPOQQR'
55
56 ### Expression 2
57
58 - Regex: '(X|Y|Z)^3(8)+(9|0)'
59 - Validation target: '^(?:X|Y|Z){3}(?:8)+(?:9|0)$'
60 - Metrics: nodes=11, depth=4, alternations=2, repeats=2, rough_paths=270
61 - Benchmark: parse_ms=0.0369, generate_ms=0.0392, validate_ms=0.0135
62 - Generated samples: 'XXX89, XXX80, XXX889, XXX880'
63 - Official examples: 'XXX89, YYY88889'
64 - Trace sample: 'YZZ89'
65
66 ### Expression 3
67
68 - Regex: '(H|I)(J|K)L*N?'
69 - Validation target: '^(?:H|I)(?:J|K)(?:L)*(?:N)?$'
70 - Metrics: nodes=11, depth=3, alternations=2, repeats=2, rough_paths=48
71 - Benchmark: parse_ms=0.027, generate_ms=0.0374, validate_ms=0.0161
72 - Generated samples: 'HJ, HJN, HJL, HJLN'
73 - Official examples: 'HJLLN, IKLLLLLL'
74 - Trace sample: 'IJLLLN'
75
76 ## Variant 3

```

```

77
78 ### Expression 1
79
80 - Regex: '0(P|Q|R)+2(3|4)'
81 - Validation target: '^0(?:?:P|Q|R))+2(?:?:3|4)$'
82 - Metrics: nodes=11, depth=4, alternations=2, repeats=1, rough_paths=726
83 - Benchmark: parse_ms=0.0347, generate_ms=0.0276, validate_ms=0.0136
84 - Generated samples: '0P23, 0P24, 0Q23, 0Q24'
85 - Official examples: '0PP23, 0QQQQ24'
86 - Trace sample: '0RPRR23'
87
88 ### Expression 2
89
90 - Regex: 'A*B(C|D|E)F(G|H|I)^2'
91 - Validation target: '^(?:?:A)*B(?:?:C|D|E)F(?:?:G|H|I)){2}$'
92 - Metrics: nodes=14, depth=4, alternations=2, repeats=2, rough_paths=162
93 - Benchmark: parse_ms=0.0355, generate_ms=0.0419, validate_ms=0.0113
94 - Generated samples: 'BCFGG, BCFGH, BCFGI, BCFHG'
95 - Official examples: 'AAABCFGG, AAAAAABDFHH'
96 - Trace sample: 'AAAABDFGG'
97
98 ### Expression 3
99
100 - Regex: 'J+K(L|M|N)*0?(P|Q)^3'
101 - Validation target: '^(?:?:J)+K(?:?:L|M|N))*(?:?:0)?(?:?:P|Q)){3}$'
102 - Metrics: nodes=15, depth=4, alternations=2, repeats=4, rough_paths=29120
103 - Benchmark: parse_ms=0.0407, generate_ms=0.0597, validate_ms=0.0154
104 - Generated samples: 'JKPPP, JKPPQ, JKQPQ, JKPPQ'
105 - Official examples: 'JJKLOPPP, JKNQQQ'
106 - Trace sample: 'JKNPQP'
107
108 ## Variant 4
109
110 ### Expression 1
111
112 - Regex: '(S|T)(U|V)W*Y+24'
113 - Validation target: '^(?:?:S|T)(?:?:U|V)(?:?:W)*(?:?:Y)+24$'
114 - Metrics: nodes=13, depth=3, alternations=2, repeats=2, rough_paths=120
115 - Benchmark: parse_ms=0.0385, generate_ms=0.0676, validate_ms=0.0482
116 - Generated samples: 'SUY24, SUYY24, SUYYYY24, SUYYYYY24'
117 - Official examples: 'SUWWY24, SVWY24'
118 - Trace sample: 'SUWWYYYY24'
119
120 ### Expression 2
121
122 - Regex: 'L(M|N)0^3P*Q(2|3)'
123 - Validation target: '^L(?:?:M|N)(?:?:0){3}(?:?:P)*Q(?:?:2|3)$'
124 - Metrics: nodes=13, depth=3, alternations=2, repeats=2, rough_paths=24
125 - Benchmark: parse_ms=0.0415, generate_ms=0.0547, validate_ms=0.0299
126 - Generated samples: 'LM000Q2, LM000Q3, LM000PQ2, LM000PQ3'
127 - Official examples: 'LM000PPPPQ2, LN000PQ3'
128 - Trace sample: 'LM000PPPPQ2'
129
130 ### Expression 3
131
132 - Regex: 'R*S(T|U|V)W(X|Y|Z)^2'
133 - Validation target: '^(?:?:R)*S(?:?:T|U|V)W(?:?:X|Y|Z)){2}$'
134 - Metrics: nodes=14, depth=4, alternations=2, repeats=2, rough_paths=162
135 - Benchmark: parse_ms=0.1118, generate_ms=0.082, validate_ms=0.0306
136 - Generated samples: 'STWXX, STWXY, STWXZ, STWYX'
137 - Official examples: 'RSTWXX, RRSUWYY'
138 - Trace sample: 'RRSUWYY'

```

G Appendix G - Analysis Module Listing

```

1 from __future__ import annotations
2
3 import json

```

```

4 import random
5 import re
6 import time
7 from pathlib import Path
8 from typing import Iterable
9
10 from .generator import explain_generation, generate_language,
    generate_random_word, to_python_regex
11 from .parser import parse_regex
12 from .variants import TASK_EXAMPLES, VARIANTS
13 from .visualization import ast_metrics
14
15
16 def _format_set(words: Iterable[str]) -> str:
17     return "{" + ", ".join(words) + "}"
18
19
20 def _resolve_targets(
21     targets: list[tuple[str, tuple[str, ...]]] | None,
22 ) -> list[tuple[str, tuple[str, ...]]]:
23     if targets is not None:
24         return targets
25
26     return [
27         (f"variant {variant_id}", VARIANTS[variant_id].expressions
28         )
29         for variant_id in sorted(VARIANTS)
30     ]
31
32 def _official_examples_for(label: str, index: int) -> tuple[str,
    ...]:
33     if not label.startswith("variant "):
34         return ()
35
36     variant_id = int(label.split()[-1])
37     bucket = TASK_EXAMPLES.get(variant_id)
38     if not bucket or index > len(bucket):
39         return ()
40     return bucket[index - 1]
41
42
43 def _ensure_sample_count(
44     expression: str,
45     words: list[str],
46     sample_count: int,
47     max_repeat: int,
48     rng: random.Random,
49 ) -> list[str]:
50     if len(words) >= sample_count:
51         return words[:sample_count]
52

```

```

53     ast = parse_regex(expression)
54     seen = set(words)
55     attempts = 0
56     max_attempts = max(100, sample_count * 50)
57
58     while len(words) < sample_count and attempts < max_attempts:
59         attempts += 1
60         candidate, _ = generate_random_word(ast, max_repeat=
61             max_repeat, rng=rng)
62         if candidate in seen:
63             continue
64         seen.add(candidate)
65         words.append(candidate)
66
67     return words[:sample_count]
68
69 def _time_ms(fn, iterations: int) -> float:
70     start = time.perf_counter()
71     for _ in range(iterations):
72         fn()
73     elapsed = time.perf_counter() - start
74     return (elapsed * 1000.0) / max(iterations, 1)
75
76
77 def _benchmark_expression(
78     expression: str,
79     *,
80     max_repeat: int,
81     max_results: int,
82     benchmark_iterations: int,
83 ) -> dict[str, float]:
84     ast = parse_regex(expression)
85     generated_once = generate_language(ast, max_repeat=max_repeat,
86         max_results=max_results)
87     pattern_text = f"^{to_python_regex(ast)}$"
88
89     parse_ms = _time_ms(lambda: parse_regex(expression),
90         benchmark_iterations)
91     generate_ms = _time_ms(
92         lambda: generate_language(ast, max_repeat=max_repeat,
93             max_results=max_results),
94         benchmark_iterations,
95     )
96     validate_ms = _time_ms(
97         lambda: [
98             re.fullmatch(pattern_text, word)
99             for word in generated_once
100         ],
101         benchmark_iterations,
102     )

```

```

101     return {
102         "parse_ms": round(parse_ms, 4),
103         "generate_ms": round(generate_ms, 4),
104         "validate_ms": round(validate_ms, 4),
105     }
106
107
108 def _expression_analysis(
109     *,
110     label: str,
111     index: int,
112     expression: str,
113     max_repeat: int,
114     max_results: int,
115     sample_count: int,
116     seed: int,
117     benchmark_iterations: int,
118 ) -> dict[str, object]:
119     ast = parse_regex(expression)
120     metrics = ast_metrics(ast, max_repeat=max_repeat)
121     python_regex = f"^{to_python_regex(ast)}$"
122
123     rng = random.Random(seed)
124     generated = generate_language(ast, max_repeat=max_repeat,
125                                   max_results=max(max_results, sample_count))
126     generated = _ensure_sample_count(expression, generated,
127                                       sample_count, max_repeat, rng)
128     trace_word, trace_steps = explain_generation(expression,
129                                                  max_repeat=max_repeat, seed=seed + 97)
130
131     return {
132         "index": index,
133         "expression": expression,
134         "python_regex": python_regex,
135         "metrics": metrics,
136         "generated": generated,
137         "official_examples": list(_official_examples_for(label,
138                                                         index)),
139         "trace": {
140             "word": trace_word,
141             "steps": [
142                 {"path": step.path, "action": step.action, "detail":
143                  step.detail}
144                 for step in trace_steps
145             ],
146         },
147         "benchmark": _benchmark_expression(
148             expression,
149             max_repeat=max_repeat,
150             max_results=max_results,
151             benchmark_iterations=benchmark_iterations,
152         ),

```

```

148     }
149
150
151 def build_lab_analysis(
152     *,
153     max_repeat: int = 5,
154     max_results: int = 200,
155     sample_count: int = 8,
156     seed: int = 42,
157     benchmark_iterations: int = 10,
158     targets: list[tuple[str, tuple[str, ...]]] | None = None,
159 ) -> dict[str, object]:
160     resolved_targets = _resolve_targets(targets)
161     variants: list[dict[str, object]] = []
162
163     expression_counter = 0
164     path_total = 0
165     max_path_entry: tuple[str, int, int] | None = None
166
167     for variant_offset, (label, expressions) in enumerate(
168         resolved_targets):
169         expression_entries: list[dict[str, object]] = []
170         for index, expression in enumerate(expressions, start=1):
171             expression_counter += 1
172             analysis = _expression_analysis(
173                 label=label,
174                 index=index,
175                 expression=expression,
176                 max_repeat=max_repeat,
177                 max_results=max_results,
178                 sample_count=sample_count,
179                 seed=seed + variant_offset * 100 + index,
180                 benchmark_iterations=benchmark_iterations,
181             )
182             rough_paths = int(analysis["metrics"]["rough_paths"])
183             # type: ignore[index]
184             path_total += rough_paths
185             if max_path_entry is None or rough_paths >
186                 max_path_entry[2]:
187                 max_path_entry = (label, index, rough_paths)
188             expression_entries.append(analysis)
189
190         variants.append({"label": label, "expressions":
191             expression_entries})
192
193     hottest = {
194         "label": max_path_entry[0] if max_path_entry else "",
195         "expression_index": max_path_entry[1] if max_path_entry
196             else 0,
197         "rough_paths": max_path_entry[2] if max_path_entry else 0,
198     }

```

```

195     return {
196         "meta": {
197             "variant_count": len(variants),
198             "expression_count": expression_counter,
199             "max_repeat": max_repeat,
200             "sample_count": sample_count,
201             "max_results": max_results,
202             "benchmark_iterations": benchmark_iterations,
203             "seed": seed,
204             "rough_path_total": path_total,
205             "heaviest_expression": hottest,
206         },
207         "variants": variants,
208     }
209
210
211 def render_terminal_transcript(analysis: dict[str, object]) -> str
212 :
213     meta = analysis["meta"] # type: ignore[index]
214     variants = analysis["variants"] # type: ignore[index]
215
216     lines = [
217         "Regular Expressions Lab 4 - analysis transcript",
218         f"seed={meta['seed']}, max_repeat={meta['max_repeat']},
219         samples={meta['sample_count']}, benchmark_iterations={
220             meta['benchmark_iterations']}",
221         ""
222     ]
223
224     for variant in variants:
225         lines.append(str(variant["label"]).upper())
226         lines.append("-" * len(str(variant["label"])))
227         for expression in variant["expressions"]:
228             bench = expression["benchmark"]
229             metrics = expression["metrics"]
230             lines.append(f"Expression {expression['index']}: {
231                 expression['expression']}")
232             lines.append(f"Generated: {_format_set(expression['
233                 generated'])}")
234             if expression["official_examples"]:
235                 lines.append(f"Task examples: {_format_set(
236                     expression['official_examples'])}")
237             lines.append(
238                 "Metrics: "
239                 f"nodes={metrics['nodes']}, depth={metrics['depth
240                     ']}", "
241                 f"alternations={metrics['alternations']}, repeats
242                     ={metrics['repeats']}", "
243                 f"rough_paths={metrics['rough_paths']}"
244             )
245             lines.append(
246                 "Benchmark: "

```

```

239         f"parse_ms={bench['parse_ms']}, generate_ms={bench
240             ['generate_ms']}", "
241         f"validate_ms={bench['validate_ms']}"
242     )
243     lines.append(f"Validation target: {expression['python_regex']}")
244     lines.append(f"Trace sample: {expression['trace']['word']}")
245     for step in expression["trace"]["steps"][:5]:
246         lines.append(
247             f"    - path={step['path']} action={step['action']} detail={step['detail']}"
248         )
249     lines.append("")
250     return "\n".join(lines).rstrip() + "\n"
251
252
253 def render_analysis_markdown(analysis: dict[str, object]) -> str:
254     meta = analysis["meta"] # type: ignore[index]
255     variants = analysis["variants"] # type: ignore[index]
256     hottest = meta["heaviest_expression"]
257
258     lines = [
259         "# Lab 4 Analysis Bundle",
260         "",
261         "## Meta",
262         "",
263         f"- Variants covered: {meta['variant_count']}",
264         f"- Expressions covered: {meta['expression_count']}",
265         f"- Repeat cap: {meta['max_repeat']}",
266         f"- Sample count per expression: {meta['sample_count']}",
267         f"- Benchmark iterations: {meta['benchmark_iterations']}",
268         f"- Heaviest bounded expression: {hottest['label']} / expression {hottest['expression_index']} ({hottest['rough_paths']} rough paths)",
269         "",
270     ]
271
272     for variant in variants:
273         lines.append(f"## {variant['label'].title()}")
274         lines.append("")
275         for expression in variant["expressions"]:
276             bench = expression["benchmark"]
277             metrics = expression["metrics"]
278             lines.append(f"### Expression {expression['index']}")
279             lines.append("")
280             lines.append(f"- Regex: '{expression['expression']}'")
281             lines.append(f"- Validation target: '{expression['python_regex']}'")
282             lines.append(

```



```

283         f"- Metrics: nodes={metrics['nodes']}, depth={
           metrics['depth']}, alternations={metrics['
           alternations']}, repeats={metrics['repeats']},
           rough_paths={metrics['rough_paths']}"
284     )
285     lines.append(
286         f"- Benchmark: parse_ms={bench['parse_ms']},
           generate_ms={bench['generate_ms']}, validate_ms
           ={bench['validate_ms']}"
287     )
288     lines.append(f"- Generated samples: '{', '.join(
           expression['generated'])}'")
289     if expression["official_examples"]:
290         lines.append(f"- Official examples: '{', '.join(
           expression['official_examples'])}'")
291     lines.append(f"- Trace sample: '{expression['trace']["
           word']}'")
292     lines.append("")
293
294     return "\n".join(lines).rstrip() + "\n"
295
296
297 def write_analysis_bundle(output_dir: str | Path, analysis: dict[
   str, object]) -> None:
298     destination = Path(output_dir)
299     destination.mkdir(parents=True, exist_ok=True)
300
301     (destination / "summary.json").write_text(
302         json.dumps(analysis, indent=2),
303         encoding="utf-8",
304     )
305     (destination / "summary.md").write_text(
306         render_analysis_markdown(analysis),
307         encoding="utf-8",
308     )
309     (destination / "terminal_transcript.txt").write_text(
310         render_terminal_transcript(analysis),
311         encoding="utf-8",
312     )

```

H Appendix H - Analysis Test Listing

```

1 from __future__ import annotations
2
3 import json
4 import subprocess
5 import sys
6 from pathlib import Path
7
8 import pytest

```

```

9
10 from src.analysis import (
11     build_lab_analysis,
12     render_terminal_transcript,
13     write_analysis_bundle,
14 )
15
16
17 def test_build_lab_analysis_covers_all_variants() -> None:
18     analysis = build_lab_analysis(
19         max_repeat=5,
20         max_results=10,
21         sample_count=4,
22         seed=11,
23         benchmark_iterations=1,
24     )
25
26     assert analysis["meta"]["variant_count"] == 4
27     assert analysis["meta"]["expression_count"] == 12
28     assert len(analysis["variants"]) == 4
29     assert analysis["variants"][0]["label"] == "variant 1"
30     assert len(analysis["variants"][0]["expressions"]) == 3
31
32
33 def
34     test_render_terminal_transcript_mentions_benchmarks_and_validation
35     () -> None:
36         analysis = build_lab_analysis(
37             max_repeat=5,
38             max_results=8,
39             sample_count=3,
40             seed=5,
41             benchmark_iterations=1,
42         )
43
44         transcript = render_terminal_transcript(analysis)
45         assert "Regular Expressions Lab 4 - analysis transcript" in
46             transcript
47         assert "Benchmark:" in transcript
48         assert "Validation target:" in transcript
49
50 def test_write_analysis_bundle_writes_expected_files(tmp_path:
51     Path) -> None:
52     analysis = build_lab_analysis(
53         max_repeat=5,
54         max_results=8,
55         sample_count=3,
56         seed=7,
57         benchmark_iterations=1,
58     )

```

```
57 write_analysis_bundle(tmp_path, analysis)
58
59 summary_json = tmp_path / "summary.json"
60 summary_md = tmp_path / "summary.md"
61 transcript_txt = tmp_path / "terminal_transcript.txt"
62
63 assert summary_json.exists()
64 assert summary_md.exists()
65 assert transcript_txt.exists()
66
67 data = json.loads(summary_json.read_text(encoding="utf-8"))
68 assert data["meta"]["variant_count"] == 4
69
70
71 def test_main_can_export_analysis_bundle(tmp_path: Path) -> None:
72     output_dir = tmp_path / "evidence"
73
74     command = [
75         sys.executable,
76         "4_regular_expressions/main.py",
77         "--variant",
78         "all",
79         "--samples",
80         "3",
81         "--max-results",
82         "8",
83         "--validate",
84         "--export-analysis-dir",
85         str(output_dir),
86         "--benchmark-iterations",
87         "1",
88     ]
89     result = subprocess.run(command, capture_output=True, text=
90                             True, check=False)
91
92     assert result.returncode == 0, result.stderr
93     assert (output_dir / "summary.json").exists()
94     assert (output_dir / "terminal_transcript.txt").exists()
```

I References

- [1] J. E. Hopcroft, R. Motwani, J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, Pearson.
- [2] M. Sipser, *Introduction to the Theory of Computation*, Cengage Learning.
- [3] Python Software Foundation, *re — Regular expression operations*, <https://docs.python.org/3/library/re.html>.

- [4] Python Software Foundation, *dataclasses* — *Data Classes*, <https://docs.python.org/3/library/dataclasses.html>.
- [5] Mermaid Documentation, <https://mermaid.js.org/>.
- [6] Project repository source code and reports contained in the current laboratory workspace.